



Politechnika Śląska

Wydział Inżynierii Materiałowej

Cyberzagrożenia

Sprawozdanie z projektu

Temat: Menadżer haseł

Prowadzący:
Dr inż. Krzysztof Daniec

Jakub Jucha
Szymon Mikoda
MIP10 sekcja 1

1. Cel projektu

Celem projektu "Password Manager" jest stworzenie aplikacji umożliwiającej:

- Bezpieczne przechowywanie haseł użytkownika.
- Automatyczne generowanie silnych haseł.
- Zarządzanie wieloma profilami użytkownika.
- Testowanie metod szyfrowania pod kątem wydajności i bezpieczeństwa.

Aplikacja została zaprojektowana z myślą o prostocie i funkcjonalności, a także zgodności z najlepszymi praktykami cyberbezpieczeństwa.

2. Funkcjonalności

Główne funkcje aplikacji:

1. Logowanie i profile użytkownika

- Możliwość tworzenia nowych profili użytkowników.
- Szyfrowanie haseł za pomocą wybranej metody (AES, HMAC).
- Zabezpieczenie hasła do profilu za pomocą funkcji skrótu SHA256.

2. Zarządzanie hasłami

- Dodawanie nowych haseł z możliwością generowania silnych haseł.
- Wyświetlanie listy zapisanych haseł.
- Usuwanie wybranych haseł.

3. Bezpieczne kopiowanie haseł

- Kopiowanie haseł do schowka z automatycznym ich usuwaniem po 30 sekundach.

4. Testowanie metod szyfrowania

- Porównanie bezpieczeństwa haseł.
- Wydajność szyfrowania i deszyfrowania.

3. Struktura aplikacji

Struktura aplikacji opiera się na podziale na widoki i klasy pomocnicze. Kluczowe elementy to:

- **Najważniejsze klasy pomocnicze:**

- ProfileInfo: Odpowiada za przechowywanie informacji o profilu użytkownika, takich jak metoda szyfrowania, hasła oraz ich zapis i odczyt z pliku.
- EncryptionManager: Zarządza procesami szyfrowania, deszyfrowania i haszowania danych, oferując wsparcie dla algorytmów AES, HMAC oraz SHA256.

- **Najważniejsze widoki:**

- LoginView: Obsługuje logowanie użytkownika oraz zarządzanie profilami.
- PasswordsView: Wyświetla tabelę zapisanych haseł, umożliwia zarządzanie nimi, w tym ich dodawanie, usuwanie i kopiowanie. Informuje o skopiowaniu hasła i wizualizuje czas do wyczyszczenia schowka.
- OptionsView: Pozwala na wybór i zmianę metody szyfrowania (AES, HMAC) dla bieżącego profilu.
- TestView: Umożliwia przeprowadzanie testów wydajności i bezpieczeństwa metod szyfrowania.

4. Technologie i biblioteki

- **Język programowania:** C#

- **Framework:** WPF (Windows Presentation Foundation)

- **Szyfrowanie i haszowanie:**

- AES (Advanced Encryption Standard)
- HMAC (Hash-based Message Authentication Code)
- SHA256 (Secure Hash Algorithm 256-bitowy)

- **Najważniejsze biblioteki systemowe:**

- System.Security.Cryptography (obsługa szyfrowania).
- System.Windows.Threading (obsługa timerów).

5. Implementacja

5.1. Zabezpieczanie hasła do profilu

Hasło użytkownika jest zabezpieczane za pomocą funkcji skrótu SHA256. Proces ten zapewnia, że:

- Hasło jest przechowywane w postaci nieodwracalnego skrótu.
- Porównywanie wprowadzonego hasła z zapisanym odbywa się poprzez ponowne haszowanie i porównanie wartości.

Za realizację tej funkcjonalności odpowiada klasa `EncryptionManager`, która zawiera metodę generującą hash z hasła użytkownika. Kluczowe znaczenie ma tutaj wykorzystanie funkcji SHA256 zapewniającej wysoką odporność na ataki brute-force. Klasa ta jest używana w module logowania oraz przy tworzeniu nowych profili w celu bezpiecznego przechowywania haseł.

```
public string Hash(string input)
{
    switch (_method)
    {
        case EncryptionMethod.SHA256:
            return HashSHA256(input);
        default:
            throw new NotSupportedException($"Metoda szyfrowania {_method} nie jest obsługiwana.");
    }
}

public bool VerifyHash(string input, string hash)
{
    return Hash(input) == hash;
}
```

5.2. Szyfrowanie danych

Dane są szyfrowane za pomocą algorytmów AES lub HMAC:

AES (Advanced Encryption Standard):

- Symetryczny algorytm szyfrowania, który wykorzystuje klucz o stałej długości.
- Dane są szyfrowane za pomocą klucza pochodzącego z hasła użytkownika.

HMAC (Hash-based Message Authentication Code):

- Wykorzystuje funkcję skrótu z kluczem do zapewnienia integralności i autentyczności danych.
- Dane są zabezpieczane za pomocą klucza pochodzącego z hasła użytkownika.

Widoczne poniżej metody szyfrujące i deszyfrujące, wywołują odpowiednie metody w zależności od wybranej metody szyfrowania.

```
public string Encrypt(string input, string key)
{
    switch (_method)
    {
        case EncryptionMethod.AES:
            return EncryptAES(input, key);
        case EncryptionMethod.HMAC:
            return EncryptHMAC(input, key);
        default:
            throw new NotSupportedException($"Metoda szyfrowania {_method} nie jest obsługiwana.");
    }
}

public string Decrypt(string encryptedInput, string key)
{
    switch (_method)
    {
        case EncryptionMethod.AES:
            return DecryptAES(encryptedInput, key);
        case EncryptionMethod.HMAC:
            return DecryptHMAC(encryptedInput, key);
        default:
            throw new NotSupportedException($"Metoda szyfrowania {_method} nie jest obsługiwana.");
    }
}
```

Metoda szyfrująca algorytmem AES:

```
private string EncryptAES(string plainText, string key)
{
    using (var aes = Aes.Create())
    {
        aes.Key = GenerateKey(key);
        aes.GenerateIV();
        byte[] iv = aes.IV;

        using var encryptor = aes.CreateEncryptor(aes.Key, aes.IV);
        byte[] plainBytes = Encoding.UTF8.GetBytes(plainText);
        byte[] cipherTextBytes = encryptor.TransformFinalBlock(plainBytes, 0, plainBytes.Length);

        string ivBase64 = Convert.ToBase64String(iv);
        string cipherTextBase64 = Convert.ToBase64String(cipherTextBytes);

        return $"{ivBase64}|{cipherTextBase64}";
    }
}
```

Metoda deszyfrująca AES:

```
private string DecryptAES(string encryptedInput, string key)
{
    var parts = encryptedInput.Split('|');
    if (parts.Length != 2)
    {
        throw new FormatException("Niepoprawny format zaszyfrowanych danych.");
    }

    string ivBase64 = parts[0];
    string cipherTextBase64 = parts[1];

    byte[] iv = Convert.FromBase64String(ivBase64);
    byte[] cipherTextBytes = Convert.FromBase64String(cipherTextBase64);

    using (var aes = Aes.Create())
    {
        aes.Key = GenerateKey(key);
        aes.IV = iv;

        using var decryptor = aes.CreateDecryptor(aes.Key, aes.IV);
        byte[] plainBytes = decryptor.TransformFinalBlock(cipherTextBytes, 0, cipherTextBytes.Length);
        return Encoding.UTF8.GetString(plainBytes);
    }
}
```

Metoda szyfrująca algorytmem HMAC:

```
private string EncryptHMAC(string plainText, string key)
{
    using (var hmac = new HMACSHA256(GenerateKey(key)))
    {
        byte[] plainBytes = Encoding.UTF8.GetBytes(plainText);
        byte[] hashBytes = hmac.ComputeHash(plainBytes);

        string hashBase64 = Convert.ToBase64String(hashBytes);
        string plainTextBase64 = Convert.ToBase64String(plainBytes);

        return $"{plainTextBase64}|{hashBase64}";
    }
}
```

Metoda deszyfrująca HMAC:

```
private string DecryptHMAC(string encryptedInput, string key)
{
    var parts = encryptedInput.Split('|');
    if (parts.Length != 2)
        throw new FormatException("Niepoprawny format zaszyfrowanych danych HMAC.");

    string plainTextBase64 = parts[0];
    string storedHashBase64 = parts[1];

    byte[] plainBytes = Convert.FromBase64String(plainTextBase64);
    byte[] storedHash = Convert.FromBase64String(storedHashBase64);

    using (var hmac = new HMACSHA256(GenerateKey(key)))
    {
        byte[] computedHash = hmac.ComputeHash(plainBytes);
        if (!computedHash.SequenceEqual(storedHash))
            throw new CryptographicException("Błąd weryfikacji HMAC. Dane mogły zostać zmodyfikowane.");
    }

    return Encoding.UTF8.GetString(plainBytes);
}
```

5.3. Generowanie hasel

Hasła generowane są w oparciu o losowe kombinacje znaków:

- Duże litery (A-Z).
- Małe litery (a-z).
- Cyfry (0-9).
- Znaki specjalne (!@#\$%^&*).

Proces generowania uwzględnia minimalną długość hasła (16 znaków) oraz różnorodność znaków. Mechanizm generowania wykorzystuje klasę Random, która zapewnia losowy wybór znaków z określonego zestawu. Generowane hasła są dodatkowo mieszane, aby uniknąć przewidywalności układu znaków.

```

private string GeneratePassword(int minLength, int maxLength)
{
    const string upperCase = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    const string lowerCase = "abcdefghijklmnopqrstuvwxyz";
    const string digits = "0123456789";
    const string specialCharacters = "!@#$%^&*()-_+=[]{}|;,:.<>?";
    const string allCharacters = upperCase + lowerCase + digits + specialCharacters;

    Random random = new Random();
    int passwordLength = random.Next(minLength, maxLength + 1);

    var password = new StringBuilder();
    password.Append(upperCase[random.Next(upperCase.Length)]);
    password.Append(lowerCase[random.Next(lowerCase.Length)]);
    password.Append(digits[random.Next(digits.Length)]);
    password.Append(specialCharacters[random.Next(specialCharacters.Length)]);

    for (int i = 4; i < passwordLength; i++)
    {
        password.Append(allCharacters[random.Next(allCharacters.Length)]);
    }

    return new string(password.ToString().OrderBy(c => random.Next()).ToArray());
}

```

5.4. Testy

Test wydajności szyfrowania

Test polega na zmierzeniu czasu potrzebnego na zaszyfrowanie i odszyfrowanie danych o rozmiarze 10 MB dla obu metod szyfrowania: AES i HMAC. Celem testu jest ocena szybkości działania algorytmów w różnych scenariuszach. Wyniki są przedstawiane w milisekundach i pozwalają na porównanie wydajności obu metod.

Test porównania haseł w zależności od długości

Test symuluje łamanie haseł za pomocą brute-force, wykorzystując różne algorytmy szyfrowania (AES, HMAC). Dwa hasła są testowane pod kątem odporności na złamanie. Wyniki pokazują, jak długo zajmuje złamanie haseł o różnej złożoności, co podkreśla znaczenie stosowania silnych haseł w praktyce.

Metoda mierząca czas potrzebny na złamanie haseł „123” oraz „1234” zaszyfrowanych z wykorzystaniem algorytmu AES oraz HMAC:

```
private void RunPasswordStrengthTest_Click(object sender, RoutedEventArgs e)
{
    string weakPassword = "123";
    string strongPassword = "1234";
    var encryptionManagerAES = new EncryptionManager(EncryptionMethod.AES);
    var encryptionManagerHMAC = new EncryptionManager(EncryptionMethod.HMAC);
    string encryptedWeakAES = encryptionManagerAES.Encrypt(weakPassword, "key");
    string encryptedStrongAES = encryptionManagerAES.Encrypt(strongPassword, "key");

    string encryptedWeakHMAC = encryptionManagerHMAC.Encrypt(weakPassword, "key");
    string encryptedStrongHMAC = encryptionManagerHMAC.Encrypt(strongPassword, "key");

    ResultsBox.Text += "Test łamania haseł użytkownika:\n";
    ResultsBox.Text += "AES:\n";
    ResultsBox.Text += TestPasswordStrength(encryptionManagerAES, encryptedWeakAES, encryptedStrongAES, "key");
    ResultsBox.Text += "HMAC:\n";
    ResultsBox.Text += TestPasswordStrength(encryptionManagerHMAC, encryptedWeakHMAC, encryptedStrongHMAC, "key");
}

private string TestPasswordStrength(EncryptionManager manager, string weakPassword, string strongPassword, string key)
{
    var sw = new Stopwatch();
    string result = "";

    sw.Start();
    BruteForce(manager, weakPassword, key, 3, 60000);
    sw.Stop();
    long weakTime = sw.ElapsedMilliseconds;

    sw.Restart();
    BruteForce(manager, strongPassword, key, 6, 60000);
    sw.Stop();
    long strongTime = sw.ElapsedMilliseconds;

    result += $"Słabsze hasło: {weakTime} ms, silniejsze hasło: {strongTime} ms\n";
    return result;
}
```


6. Wyniki testów

6.1. Testy wydajności

Test został wykonany trzykrotnie, kolejne wyniki oddziela znak „|”.

Metoda szyfrowania	Szyfrowanie (ms)	Deszyfrowanie (ms)
AES	36 29 26	44 28 38
HMAC	34 18 34	36 28 51

6.2. Porównanie słabego i silnego hasła

Test został wykonany trzykrotnie, kolejne wyniki oddziela znak „|”.

Hasło	AES (ms)	HMAC (ms)
123	657 492 473	354 358 386
1234	22810 23439 30117	29546 29574 23883

7. Przypadki użycia

7.1. Zakładanie konta

- Opis: Użytkownik zakłada konto poprzez podanie głównego hasła, które będzie służyło do uwierzytelniania w aplikacji.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik uruchamia aplikację.
 2. Wybiera opcję „Nowy profil”.
 3. Podaje hasło główne i potwierdza je.
 4. Aplikacja informuje o pomyślnym założeniu konta.

7.2. Logowanie

- Opis: Użytkownik loguje się za pomocą głównego hasła.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik uruchamia aplikację.
 2. Wybiera profil.
 3. Wprowadza hasło główne i wciska „Zaloguj”.
 4. Aplikacja uwierzytelnia użytkownika i umożliwia dostęp do bazy haseł.
- Przebieg alternatywny:
 - W przypadku błędnego hasła aplikacja wyświetla komunikat o nieudanym logowaniu.

7.3. Dodawanie hasła

- Opis: Użytkownik dodaje nowe hasło do swojej bazy.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik po zalogowaniu wybiera opcję „Dodaj” na głównym ekranie.
 2. Podaje nazwę platformy, hasło i opcjonalny opis.
 3. Aplikacja zapisuje hasło w odpowiednim pliku z rozszerzeniem .psmgr.

7.4. Wybór metody szyfrowania

- Opis: Użytkownik wybiera metodę szyfrowania dla przechowywanych haseł.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik przechodzi do ustawień aplikacji.
 2. Wybiera jedną z dostępnych metod szyfrowania.
 3. Aplikacja zapisuje preferencję szyfrowania i ponownie szyfruje hasła.

7.5. Weryfikacja hasła

- Opis: Aplikacja sprawdza bezpieczeństwo dodanego hasła.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik dodaje nowe hasło.
 2. Aplikacja analizuje hasło pod kątem złożoności i długości.
 3. Aplikacja informuje użytkownika o wynikach weryfikacji.

7.6. Generowanie hasła

- Opis: Aplikacja generuje silne hasło zgodne z ustalonymi kryteriami.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik podczas dodawania nowego hasła wybiera opcję „Wygeneruj hasło”.
 2. Aplikacja generuje hasło o odpowiednim poziomie skomplikowania.
 3. Użytkownik decyduje, czy zapisać hasło.

7.8. Kopiowanie hasła do schowka

- Opis: Użytkownik kopiuje hasło do schowka, które jest automatycznie usuwane po określonym czasie.
- Aktor: Użytkownik.
- Przebieg główny:
 1. Użytkownik wybiera hasło w aplikacji.
 2. Kliknięciem kopiuje je do schowka.
 3. Aplikacja ustawia timer na automatyczne usunięcie hasła ze schowka.